Evolving Neural Networks through a Reverse Encoding Tree

Haoling Zhang
Institute of Biochemistry
BGI-Shenzhen
Shenzhen, Guangdong, China
zhanghaoling@genomics.cn

Chao-Han Huck Yang
School of ECE
Georgia Institute of Technology
Atlanta, GA, USA
huckiyang@gatech.edu

Hector Zenil
Algorithmic Dynamics Lab &
Oxford Immune Algorithmics
U.K. & Sweden
hector.zenil@cs.ox.ac.uk

Narsis A. Kiani
Algorithmic Dynamics Lab
Karolinska Institute
Stockholm, Sweden
narsis.kiani@ki.se

Yue Shen
Institute of Biochemistry
BGI-Shenzhen
Shenzhen, Guangdong, China
shenyue@genomics.cn

Jesper N. Tegner*
Living Systems Lab, BESE, CEMSE
King Abdullah University of Science and Technology
Thuwal 23955, Saudi Arabia
jesper.tegner@kaust.edu.sa

arXiv:2002.00539v2 [cs.NE] 31 Mar 2020

***Abstract*—NeuroEvolution is one of the most competitive evolutionary learning strategies for designing novel neural networks for use in specific tasks, such as logic circuit design and digital gaming. However, the application of benchmark methods such as the NeuroEvolution of Augmenting Topologies (NEAT) remains a challenge, in terms of their computational cost and search time inefficiency. This paper advances a method which incorporates a type of topological edge coding, named Reverse Encoding Tree (RET), for evolving scalable neural networks efficiently. Using RET, two types of approaches – NEAT with Binary search encoding (Bi-NEAT) and NEAT with Golden-Section search encoding (GS-NEAT) – have been designed to solve problems in benchmark continuous learning environments such as logic gates, Cartpole, and Lunar Lander, and tested against classical NEAT and FS-NEAT as baselines. Additionally, we conduct a robustness test to evaluate the resilience of the proposed NEAT approaches. The results show that the two proposed approaches deliver an improved performance, characterized by (1) a higher accumulated reward within a finite number of time steps; (2) using fewer episodes to solve problems in targeted environments, and (3) maintaining adaptive robustness under noisy perturbations, which outperform the baselines in all tested cases. Our analysis also demonstrates that RET expends potential future research directions in dynamic environments. Code is available from https://github.com/HaolingZHANG/ReverseEncodingTree.**

***Index Terms*—NeuroEvolution, Evolutionary Strategy, Continuous Learning, and Edge Encoding**

I. Introduction

NeuroEvolution (NE) is a class of methods for evolving artificial neural networks through evolutionary strategies [1], [2]. The main advantage of NE is that it allows learning under conditions of sparse feedback. In addition, the population-based process makes for good parallelism [3], without the computational requirement of back-propagation. The evolutionary process of NE is modifying connection weights in the fixed network topology (individuals) [4] by calculating their fitnesses and evaluating the relationship between individuals.

Recent studies [4]–[6] show that the trade-off between protecting topological innovations and promoting evolutionary speed is also a challenge. The evolutionary process from the initial to the final individual is difficult to control accurately. Genetic Algorithm (GA) using Speciation Strategies [7] allow a meaningful application of the crossover operation and protect topological innovations, avoiding premature disappearance. The distribution estimation algorithms, such as Population-Based Incremental Learning [8] (PBIL), represents a different way of describing the distribution information of candidate topologies of neural networks in the search space, i.e. by establishing a probability model. The Covariance Matrix Adaptation Evolution Strategy [9] (CMA-ES) further explains the correlations between the parameters of a targeted fitness function, correlations which significantly influence the time taken to find a suitable control strategy [10]. Safe Mutation [6] can scale the degree of mutation of each weight, and thereby expand the scope of domains amenable to NE. In this study, a mapping relationship, based on constraining the topological scale, is set up between its features and fitness, in order to explore how the evolutionary strategy influences the population in a restricted search space. The limitation of this scale serves to prevent unrestricted expansion of structures of neural network during evolutionary process. On the restricted topological scale, all neural networks that can be generated by their features have achieved fitness through specific tasks. The location of the specific neural network is the location of its features on the constrained topological scale. In this situation, the location of the nearest two neural networks can be regarded as infinitesimal, and the function made up of all locations is continuous. We define the location of an individual, which created by its feature matrix, as the input of the function, and its fitness as the output. Together, all locations form a complex and smooth fitness landscape [11].

In this fitness landscape, all evolutionary processes of the topology of the neural network can be regarded as processes of tree-based searching, like random forest [12]. The initial population can be regarded as the root nodes, and the population of each generation can be regarded as the branch nodes of each layer. Based on the current population or other population information (such as the probability matrix), more representative or better nodes will be identified in the next layer and used as individuals in the next generation. Interestingly, certain classical search methods have attracted our attention. Some

global search methods, like Binary Search [13] and Golden-Section Search [14], are not merely of use in finding extreme values in uni-modal functions, but have also shown promise when used in other fields [15], [16]. The search processes of the above global search method are similar to the reverse process of tree-based searching [17]. The final (root) node is dependent on the elite leaf or branch nodes in each layer, as the topology of the final neural network is influenced by the features of the elite topology of each generation.

Based on the reverse process of tree-based searching (as the evolutionary strategy), we design two specific strategies in the fitness landscape, named NEAT with a reverse binary encoding tree (Bi-NEAT) and NEAT with a reverse golden-section [18] encoding tree (GS-NEAT). In addition, the correlation coefficient [19] is used to analyze the degree of exploration of multiple sub-clusters [20] in the fitness landscape formed by each generation of individuals. It effectively prevents the population from falling into the optimal local solution due to over-rapid evolution. The evolution speed of NEAT and FS-NEAT (as the baselines) and our proposed strategies are discussed in the logic operations and continuous control gaming benchmark in the OpenAI-gym [21]. These strategies have also passed different levels and types of noise tests to establish their robustness. We reach the following conclusions: (1) Bi-NEAT, and GS-NEAT can improve the evolutionary efficiency of the population in NE; (2) Bi-NEAT and GS-NEAT show a high degree of robustness when subject to noise; (3) Bi-NEAT and GS-NEAT usually yield simpler topologies than the baselines.

II. Related Work

In this study, we introduce a search method into NeuroEvolution, and extract features in the neural network for the purpose of encoding the feature matrix. Therefore we devote this section to brief descriptions of the following three topics: (1) Evolutionary Strategies in NeuroEvolution; (2) Search Methods; and (3) Network Coding Methods.

A. Evolutionary Strategies in NeuroEvolution

NeuroEvolution (NE) is a combination of Artificial Neural Network and Evolutionary Strategy. Preventing crossovering the topologies efficiently, protecting new topological innovation, and keeping topological structure simple are three core problems faced in dealing with the Topology and WEight of Artificial Neural Network (TWEANN) system [22]. In recent years, many effective ideas have been introduced into NE. An important breakthrough came in the form of NEAT [4], [5], which protects innovation by using a distance metric to separate networks in a population into species, while controlling crossover with innovation numbers. However, the evolutionary efficiency of the population in each generation cannot be guaranteed. In order to guarantee the evolutionary efficiency of NE, three research paths have been devised: (1) the replacement of the original speciation strategy with a new speciation strategy [7]; (2) the introduction of more effective evolutionary strategies [6], [8], [10]; (3) the use of novel topological structures [23].

Certainly, modifying the structure and/or weight involves much more than the feature information of ANN itself. The above improvements make it challenging to prevent the modification of all features. Furthermore, the complexity of the topology required for obtaining the required ANN is unlimited, which means that the topological structure of ANN will not be necessarily simple.

B. Search Methods

In the field of Computer Science, search trees, such as the Binary Search tree [24], are based on the idea of divide and conquer method. They are often used to solve extrema in uni-modal arrays or find specific values in sorted arrays.

Recently, some improved search trees have also been used to solve extrema in multi-modal or other optimization fields [15], [16], [25], [26]. These search trees, such as Binary Search [13], Golden-Section Search [14], and Golden-Section SINE Search [26] complete complex tasks by combining with population [27] or other strategies [15]. They make the whole population develop more accurately with geometric searching. In the field of multi-modal searching [28], they increase global optimization ability by estimating and abandoning small peaks. Therefore using tree-based search has the potential to improve evolutionary efficiency. In addition, tree-based searches have a strong resistance to environmental noise [29], where position of optimum point would be generated by a sampling-based distribution to enhance interference on noisy observation.

Given the crossover operation of topologies, some search methods have spurred an interest in enhancing the precision of such crossover operations, thus opening up an interesting avenue for the introduction of search trees into NE.

C. Network Coding Methods

At the stage of direct coding, the encoding rule of ANN is to convert it into a genotype [4]. In order to generate large-scale, functional, and complex ANN, some indirect coding [30], [31] techniques have been proposed. However, they are not efficient enough for the evolution of local networks, because decreasing the granularity of coordinates leads to a decrease in resolution [32]. The above encoding is a kind of cellular encoding [33], which uses chromosomes or genotypes consisting of trees of node operators to evolve a graph.

Edge encoding [34], which is different from cellular encoding, grows a graph by modifying its edges, and thus has a different encoding bias than the genetic search process. When naturally evolving network topologies, edge encoding is often better than cellular encoding [34]. Edge encoding can use adjacency matrices as representational tools [35]. An adjacency matrix represents a graph with a type of square matrix where each element represents an edge. The corresponding nodes connected by weight are indicated by the row and column of the edge in the matrix.

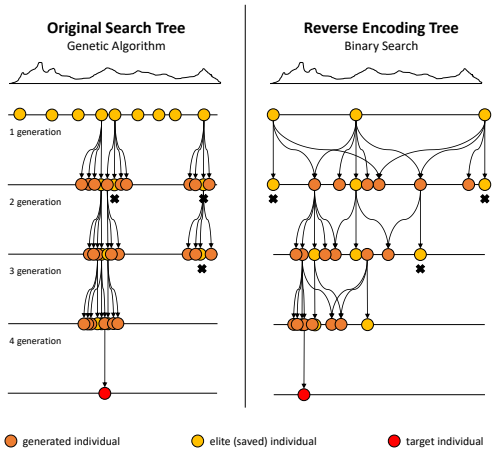

Fig. 3. Illustration of two types of proposed tree-based network encoding. In addition to original generated way of novel individuals, RET provides discovered opportunities for potential elite individuals on the global range, using edge encoding from every two elite individuals.

The search process of RET is divided into two parts: (1) the creation of a nearby individual from the specified parent individual by the original frame of NEAT:

$$\boldsymbol{I}_{\mathrm{N}}^{(i)} = \mathrm{neat}(\boldsymbol{I}^{(i)}) \tag{3}$$

(2) the creation of a global individual from the two specified parent individuals or feature matrices:

$$\boldsymbol{I}_{\mathrm{G}}^{(i,j)} = \begin{cases} \boldsymbol{I}_{\mathrm{Bi}}^{(i,j)} \\ \boldsymbol{I}_{\mathrm{GS}}^{(i,j)} \end{cases} \tag{4}$$

And this work includes binary search (Eq. 5) and golden section search (Eq. 6).

$$\boldsymbol{I}_{\mathrm{Bi}}^{(i,j)} = \mathrm{create}(\frac{\boldsymbol{f}^{(i)} + \boldsymbol{f}^{(j)}}{2}) \tag{5}$$

$$\boldsymbol{I}_{\mathrm{GS}}^{(i,j)} = \mathrm{create}(\frac{(\sqrt{5}-1) \times \boldsymbol{f}^{(i)} + (3-\sqrt{5}) \times \boldsymbol{f}^{(j)}}{2}) \tag{6}$$

C. Analysis of Evolvability

We further propose an efficient, unsupervised learning method for analyzing the network seeds generated. The motivation for clustering the population [36] based on the similarity of individuals is to explore the evolvability of each type of individual set after protecting topology innovations. The current population is divided into p clusters for understanding the local situation of the landscape generated by the current population. Many clustering methods can be used in this strategy. We compared K-means++ [40], Spectral Clustering [41], and Birch Clustering [42], and selected the most advanced, K-means++, thus:

$$\begin{aligned} \boldsymbol{S} &= \mathrm{cluster}(\boldsymbol{P}) \\ &= \arg\min \sum_{i=1}^{p} \sum_{\boldsymbol{I} \in \boldsymbol{S}^{(i)}} d(\boldsymbol{I}.\boldsymbol{f}, \boldsymbol{c}^{(i)}) \end{aligned} \tag{7}$$

where $\boldsymbol{S}$ is the set of p clusters, $\boldsymbol{S}^{(i)}$ is the i^{th} cluster, and $\boldsymbol{c}^{(i)}$ is the center of the i^{th} cluster. The optimal individual $\boldsymbol{I}^{(i.m)}$ in the i^{th} cluster can be obtained by comparing the fitness of each individual:

$$\boldsymbol{I}^{(i.m)} = \arg\max_{\boldsymbol{I} \in \boldsymbol{S}^{(i)}} \boldsymbol{I}.r \tag{8}$$

where r is the fitness of the individual. The set of saved individuals collects the optimal individual in every cluster:

$$\boldsymbol{P}_{\mathrm{saved}} = \langle \boldsymbol{I}^{(i.m)}, i \in \mathbb{N}^{+} \cap i \leq p \rangle \tag{9}$$

The correlation coefficient (ρ) of distance from the optimal position of the individual and fitness for all the individuals in each cluster is calculated, to describe the situation of each cluster:

$$\begin{aligned} \rho^{(i)} &= \mathrm{corr}(\boldsymbol{I}^{(i.m)}, \boldsymbol{S}^{(i)}) \\ &= \mathrm{corr}(d(\boldsymbol{I}^{(i.m)}.\boldsymbol{f}, \boldsymbol{I}.\boldsymbol{f}) \Longleftrightarrow \boldsymbol{I}.r, \boldsymbol{I} \in \boldsymbol{S}^{(i)}) \end{aligned} \tag{10}$$

For the local $fitness$ landscape of a single maximum value, the distance and fitness show a negative correlation (positive ρ), ρ will reach -1. If the landscape is complex (negative ρ), the relationship between distance and fitness is not significant. Two types of ρ are shown in Fig. 4.

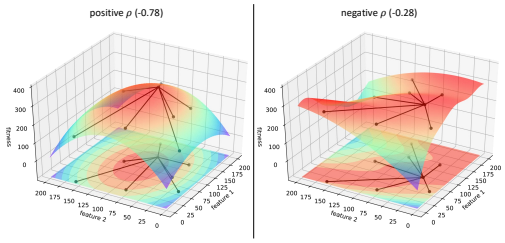

Fig. 4. Two types of ρ in a cluster

RET's operation occurs between each of the two clusters:

$$\begin{aligned} \boldsymbol{P}_{\mathrm{novel}} &= \langle \sum_{i=1}^{p-1} \sum_{j=i+1}^{p} \mathrm{ret}_{ij} \rangle \\ &= \langle \sum_{i=1}^{p-1} \sum_{j=i+1}^{p} \mathrm{ret}([\boldsymbol{S}^{(i)}, \rho^{(i)}], [\boldsymbol{S}^{(j)}, \rho^{(j)}]) \rangle \end{aligned} \tag{11}$$

The operation selection is dependent on the optimal individuals and the correlation coefficients of the two specified clusters. Therefore, the number of novel individuals is less than or equal to $p^2 - p$. We assume that if $\rho^{(i)} \leq -0.5$, i^{th} cluster has been explored fully, or its local $fitness$ landscape is simple. When $\boldsymbol{I}^{(i.m)} > \boldsymbol{I}^{(j.m)}$, the operation selection in each comparison is:

$$\mathrm{ret}_{ij} = \begin{cases} \boldsymbol{I}_{\mathrm{N}}^{(i.m)}, \boldsymbol{c}_{\mathrm{G}}^{(i,j)} & \rho^{(i)} \leq -\frac{1}{2} \cap \rho^{(j)} \leq -\frac{1}{2} \\ \boldsymbol{I}_{\mathrm{N}}^{(i.m)}, \boldsymbol{I}_{\mathrm{N}}^{(i.m)} & \rho^{(i)} > -\frac{1}{2} \cap \rho^{(j)} \leq -\frac{1}{2} \\ \boldsymbol{I}_{\mathrm{N}}^{(i.m)}, \boldsymbol{I}_{\mathrm{N}}^{(j.m)} & \rho^{(i)} \leq -\frac{1}{2} \cap \rho^{(j)} > -\frac{1}{2} \\ \boldsymbol{I}_{\mathrm{N}}^{(i.m)}, \boldsymbol{I}_{\mathrm{N}}^{(j.m)} & \rho^{(i)} > -\frac{1}{2} \cap \rho^{(j)} > -\frac{1}{2} \end{cases} \tag{12}$$

where $\boldsymbol{c}_{\mathrm{G}}^{(i,j)}$ is the novel individual created by two centers of the specified cluster.

In summary, our proposed evolutionary strategy uses RET based on the local $fitness$ landscape to evolve the feature matrix of individuals in the population. The pseudo-code of this evolutionary process is shown in Alg. 1.

Algorithm 1 Evolution process of NEAT with RET

Input: d_i, d_m, p, m
Output: $\boldsymbol{I}$

$\boldsymbol{P} \leftarrow \emptyset$
while $\mathrm{len}(\boldsymbol{P}) < p$ **do**
  $\boldsymbol{I} \leftarrow \mathrm{create}(\boldsymbol{f})$ where $\boldsymbol{f} \in \Theta$
  if $\mathrm{check}(d_i, \boldsymbol{I}, \boldsymbol{P})$ **then**
    $\boldsymbol{P} \leftarrow \boldsymbol{P} + \boldsymbol{I}$
  end if
end while
while True **do**
  calculate r in each $\boldsymbol{I}$ where $\boldsymbol{I} \in \boldsymbol{P}$
  if one of $\boldsymbol{I}.r$ meet fitness threshold **then**
    return $\boldsymbol{I}$ where $\boldsymbol{I}.r$ meet fitness threshold
  end if
  $\boldsymbol{S} \leftarrow \mathrm{cluster}(\boldsymbol{P})$, $\boldsymbol{P}_{\mathrm{saved}} \leftarrow \emptyset$, $\boldsymbol{P}_{\mathrm{novel}} \leftarrow \emptyset$
  for $i = 1 \rightarrow p$ **do**
    $\rho^{(i)} \leftarrow \mathrm{corr}(\boldsymbol{I}^{(i.m)}, \boldsymbol{S}^{(i)})$
  end for
  for $i = 1 \rightarrow p$ **do**
    if $\mathrm{check}(d_m, \boldsymbol{I}^{(i.m)}, \boldsymbol{P}_{\mathrm{saved}})$ **then**
      $\boldsymbol{P}_{\mathrm{saved}} \leftarrow \boldsymbol{P}_{\mathrm{saved}} + \boldsymbol{I}^{(i.m)}$
    end if
  end for
  for $i = 1 \rightarrow (p-1)$ **do**
    for $j = i+1 \rightarrow p$ **do**
      $\boldsymbol{I}^{(1)}, \boldsymbol{I}^{(2)} \leftarrow \mathrm{ret}([\boldsymbol{S}^{(i)}, \rho^{(i)}], [\boldsymbol{S}^{(j)}, \rho^{(j)}])$
      if $\mathrm{check}(d_m, \boldsymbol{I}^{(1)}, \boldsymbol{P}_{\mathrm{saved}} + \boldsymbol{P}_{\mathrm{novel}})$ **then**
        $\boldsymbol{P}_{\mathrm{novel}} \leftarrow \boldsymbol{P}_{\mathrm{novel}} + \boldsymbol{I}^{(1)}$
      end if
      if $\mathrm{check}(d_m, \boldsymbol{I}^{(2)}, \boldsymbol{P}_{\mathrm{saved}} + \boldsymbol{P}_{\mathrm{novel}})$ **then**
        $\boldsymbol{P}_{\mathrm{novel}} \leftarrow \boldsymbol{P}_{\mathrm{novel}} + \boldsymbol{I}^{(2)}$
      end if
    end for
  end for
  $\boldsymbol{P} \leftarrow \boldsymbol{P}_{\mathrm{saved}} + \boldsymbol{P}_{\mathrm{novel}}$
end while

IV. Experiments

In order to verify whether NE based on tree search can improve evolutionary efficiency and fight against environmental noise effectively, we designed a two-part experiment: (1) We explore the effect of our proposed strategies and the baseline strategies in classical tasks, such as the logic gate; (2) We explore the effect of our proposed strategies and the baseline strategies in one of the classical tasks (Cartpole-v0) under different noise conditions as a robustness test for continuous learning [43].

A. Logic Gate Representative

The two-input symbolic logic gate, XOR, is one of the benchmark environments in the NEAT setting. The task is to evolve a network that distinguishes a correct Boolean output from $\{True(1), False(0)\}$. The initial reward is 4.0, and the reward will decrease by the Euclidean distance between ideal outputs and actual outputs. We select a higher targeted reward of 3.999 to tackle this environment. In addition, we add three kinds of additional logic gate, IMPLY, NAND, and NOR, to explore algorithm performance with different task complexities. The complete hyper-parameter setting in the logical experiments is as shown in Tab. I. To enhance the reproducibility of our work, select the XOR environment from the most popular **neat-python** [1] package and open-source our implementation in the supplementary material.

TABLE I
Hyper-parameters in the logical experiments.

hyper-parameter	value
iteration	1000
fitness threshold	3.999
evolution size	132
activation	sigmoid

Iteration represents the repeated number on one experiment, the metrics is coherent with previous works [7], [23]. "evolution size" describes the number of individuals need to be evolved structure and calculated fitness in each generation, which is different from population size [4].

B. Continuous Control Environment

Fig. 5. Illustration of a continuous control environment utilized as our task: (1) Cartpole-v1 [21] and (2) Cartpole subject to a background perturbation of Gaussian noise.

Our testing platforms were based on OpenAI Gym [21], well adapted for building a baseline for continuous control. **Cartpole:** As a classical continuous control environment [44], the Cartpole-v0 [21] environment is controlled by bringing to bear a force of $+1$ or -1 to the cart. A pendulum starts upright, and the goal is to prevent it from toppling over. An accumulated reward of $+1$ would be given before a terminated environment (e.g., falling 15 degrees from vertical, or a cart shifting more than 2.4 units from the center). As experimental settings, we select 1000 iteration, and use relu activation for neural network output to select an adaptive action in Tab. II. To solve the problem, we conduct and fine-tune both

[1] https://neat-python.readthedocs.io/en/latest/xor_example.html

NEAT and FS-NEAT as baseline results for accessing targeted accumulated rewards of 195.0 in 200 episode steps [21].

Here, we have improved the requirements of the fitness threshold (499.5 rewards in 500 episode steps) and normalized the fitness threshold as $\frac{\text{rewards}}{\text{episode steps}}$. See Tab. II.

TABLE II
HYPER-PARAMETERS IN THE CARTPOLE V0.

hyper-parameter	value
iteration	1000
fitness threshold	0.999
evolution size	6
activation	relu
episode steps	500
episode generation	20

C. Gaming Environment

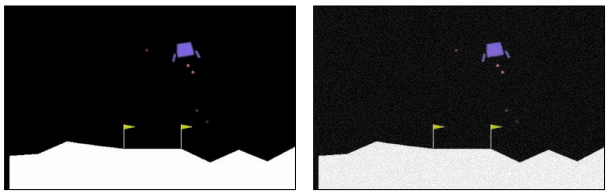

Fig. 6. Illustration of a 2D gaming environment utilized as our task: (1) Lunar Lander-v2 from the OpenAI Gym [21] and (2) Lunar Lander-v2 subject to a background perturbation of Gaussian noise.

Lunar Lander: We utilize a box-2d gaming environment, lunar lander-v2 as shown in Fig. 6, from OpenAI Gym [21]. The objective of the game is to navigate the lunar lander spaceship to a targeted landing site on the ground without collision, using two lateral thrusters and a rocket engine. Each episode lasts at most 1000 steps and runs at 50 frames per second. An episode ends when the lander flies out of borders, remains stationary on the ground, or when time is expired. A collection of six discrete **actions** that correspond to the $\{left, right\}$ off steering commands and $\{on, off\}$ main engine settings. The state, $s \in R^8$, is an eight-dimensional vector that continuously records and encodes the landers position, velocity, angle, angular velocity, and indicators for the contact between the legs of the vehicle and the ground. For the experiment, we run 1000 iterations for the Cartpole-v0 setting with details in Tab. III.

TABLE III
HYPER-PARAMETERS IN THE LUNARLANDER V2.

hyper-parameter	value
iteration	1000
fitness threshold	-0.2
evolution size	20
activation	relu
episode steps	100
episode generation	2

D. Robustness

One of the remain challenges for continuous learning is noisy observation [45] in the real-world. We further evaluate the Cartpole-v0 [21] with a shared noisy benchmark from the bsuite [45]. The hyper-parameter setting is shown in Tab. IV.

Gaussian Noise Gaussian noise or white noise is a common interference in sensory data. The interfered observation becomes $\mathcal{S}_t = s_t + n_t$ with a Gaussian noise n_t. We set up the Gaussian noise by computing the variance of all recorded states with a mean of zero.

Reverse Noise Reverse noise maps the original observation data reversely. Reverse noise is a noise evaluation for sensitivity tests with a higher L2-norm similarity but should affect the learning behavior on the physical observation. Reverse observation has been used in the continuous learning framework for communication system [46] to test its robustness against jamming attacks. Since 100% of the noise environment is consistent with a noise-free environment, we dilute the noise level to the original 50% (as dilution coefficient in Reverse).

TABLE IV
HYPER-PARAMETERS IN THE NOISE EXPERIMENTS.

hyper-parameter	value
benchmark task	CartPole v0
iteration	1000
evolution size	6
activation	relu
episode steps	300
episode generation	2
normal maximum	0.10
normal minimum	0.05
dilution coefficient in Reverse	50%
peak in Gaussian	0.20

E. Baselines

Here we take NEAT and FS-NEAT as baselines. The weight of connection and bias of node are the default settings in the example of **neat-python**.

V. RESULTS

After running 1000 iterations for each method in the logical experiments, continuous control and game experiments, and noise attack experiments, we obtained the results shown in Tab. V, Tab. VI, and Fig. 7. The evolutionary process across all the methods has the same fitness number in each generation. Therefore the comparison of average end generation is the same as the comparison of calculation times for the neural network in the evolutionary process.

After restraining the influence of hyper-parameters, the tasks from Tab. V describe the influence of task complexity on evolutionary strategies. The results show that with the increase in task difficulty, our algorithm can make the population evolve faster. In the IMPLY task, the difference between the average end generation is 1 to 2 generations. When the average of end generations in XOR tasks is counted, the gap between our proposed strategies and the baselines widens to nearly 20 generations. Additionally, the average node number in the

final neural network and the task complexity seem to have a potentially positive correlation.

TABLE V
RESULT STATISTICS IN THE EXPERIMENTS OF LOGIC GATES.

task	method	fall rate	Avg.gen	StDev.gen
IMPLY	NEAT	0.1%	7.03	1.96
	FS-NEAT	0.0%	6.35	2.21
	Bi-NEAT	0.0%	5.00	2.50
	GS-NEAT	0.0%	5.82	2.88
NAND	NEAT	0.1%	13.02	3.87
	FS-NEAT	0.0%	12.50	4.34
	Bi-NEAT	0.0%	10.26	5.26
	GS-NEAT	0.0%	11.74	5.82
NOR	NEAT	0.1%	13.13	4.18
	FS-NEAT	0.0%	12.83	4.58
	Bi-NEAT	0.0%	10.60	5.64
	GS-NEAT	0.0%	11.86	6.29
XOR	NEAT	0.1%	103.42	56.02
	FS-NEAT	0.1%	101.19	50.72
	Bi-NEAT	0.0%	84.15	30.58
	GS-NEAT	0.0%	88.11	36.13

The tasks in the continuous control and game environments Bi-NEAT and GS-NEAT still show amazing potential. See Tab. VI. Unlike in the case of the logical experiments, the results show that the two proposed strategies are superior both in terms of evolutionary speed and stability. The enhanced evolutionary speed is reflected in the fact that the baselines require two to three times the average end generation as our strategies for the tested tasks. In addition, the smaller standard variance of end generation shows the evolutionary stability of our strategies.

TABLE VI
RESULT STATISTICS IN THE COMPLEX EXPERIMENTS.

task	method	fall rate	Avg.gen	StDev.gen
CartPole v0	NEAT	26.5%	147.33	99.16
	FS-NEAT	4.8%	72.86	85.08
	Bi-NEAT	0.0%	29.35	18.86
	GS-NEAT	0.0%	31.95	22.56
LunarLander v2	NEAT	4.9%	144.21	111.87
	FS-NEAT	3.3%	152.91	108.61
	Bi-NEAT	0.0%	48.66	44.57
	GS-NEAT	0.0%	44.57	50.29

As shown in Fig. 7, the evolutionary strategies based on RET show strong robustness in the face of noise. With the increase in noise level, the fail rate of all the tested strategies increases gradually. In most cases, the baselines show a higher fail rate than our strategies. The robustness under noisy perturbation of Bi-NEAT and GS-NEAT are basically the same, the difference between the two is less than 0.4%. In the task with the low noise level, our strategies have a fail rate of one, as compared to dozens for the baselines. However, in a few cases with high noise levels, all the strategies are unable to achieve results.

VI. DISCUSSION

In general, with the same fitness number of population, Bi-NEAT and GS-NEAT show better performance by ending up with fewer generations than NEAT for the symbolic logic,

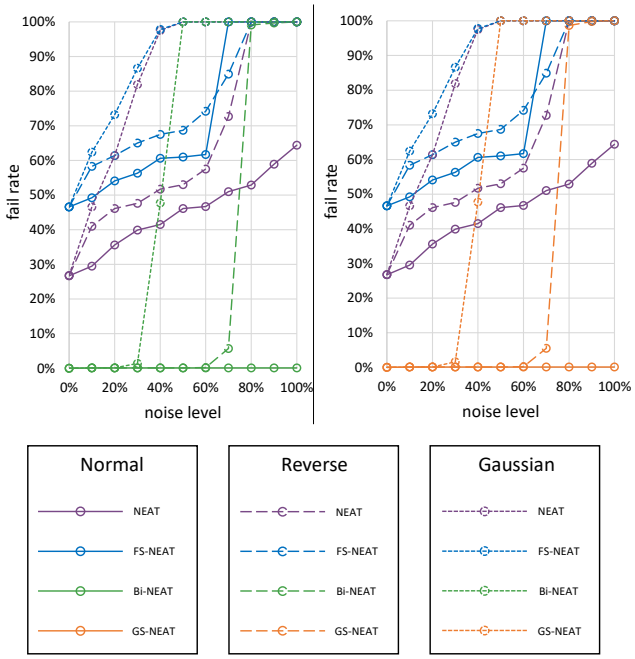

Fig. 7. Robust evaluation in CartPole-v0 thorough noisy observations included: reverse and Gaussian perturbations in Sec. IV-D.

continuous control, and 2D gaming as the benchmark environments in this study. Our proposed strategies are also superior in the tested tasks with incremental noisy observation. We conclude than they are robust in the face of noise attacks, able to deal easily with sparse and noisy data.

More interestingly, the performance nuances of Bi-NEAT and GS-NEAT in different tasks also attracted our attention. It is clear that Bi-NEAT is better than GS-NEAT in all tasks without noise. Our preliminary conclusion is that evolutionary speed is affected by the $fitness$ landscape of different tasks, because the local peak of the landscape is usually small and sharp, as implied by the process data. Another interesting point we observed is that GS-NEAT usually fares better than Bi-NEAT in the noise test. Further efforts could be performed to investigate the underneath mechanism and theoretical bounds.

VII. CONCLUSION

This paper introduced two specific evolutionary strategies based on RET for NE, namely Bi-NEAT and GS-NEAT. The experiments with logic gates, Cartpole, and Lunar Lander show that Bi-NEAT and GS-NEAT have faster evolutionary speeds and greater stability than NEAT and FS-NEAT (baselines). The noise test in Cartpole also shows stronger robustness than the baselines.

The influence of evolutionary speed, stability, and robustness of the whole strategy on the location selection of new topology nodes [47] is worth further study on biological systems. An assumption to validate is that this location selection can be adaptive vis-a-vis the landscape [48] of generation.

ACKNOWLEDGMENTS

This work was initiated by Living Systems Laboratory at King Abdullah University of Science and Technology (KAUST) lead by Prof. Jesper Tegner and supported by funds from KAUST. Chao-Han Huck Yang was supported by the Visiting Student Research Program (VSRP) from KAUST.

REFERENCES

[1] K. O. Stanley, J. Clune, J. Lehman, and R. Miikkulainen, "Designing neural networks through neuroevolution," *Nature Machine Intelligence*, vol. 1, no. 1, pp. 24–35, 2019.

[2] A. M. Zador, "A critique of pure learning and what artificial neural networks can learn from animal brains," *Nature Communications*, vol. 10, no. 1, pp. 1–7, 2019.

[3] J. Lehman and R. Miikkulainen, "Neuroevolution," *Scholarpedia*, vol. 8, no. 6, p. 30977, 2013.

[4] K. O. Stanley and R. Miikkulainen, "Evolving neural networks through augmenting topologies," *Evolutionary computation*, vol. 10, no. 2, pp. 99–127, 2002.

[5] S. Whiteson, P. Stone, K. O. Stanley, R. Miikkulainen, and N. Kohl, "Automatic feature selection in neuroevolution," in *Proceedings of the 7th annual conference on Genetic and evolutionary computation*. ACM, 2005, pp. 1225–1232.

[6] J. Lehman, J. Chen, J. Clune, and K. O. Stanley, "Safe mutations for deep and recurrent neural networks through output gradients," in *Proceedings of the Genetic and Evolutionary Computation Conference*. ACM, 2018, pp. 117–124.

[7] J. S. Knapp and G. L. Peterson, "Natural evolution speciation for neat," in *2019 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2019, pp. 1487–1493.

[8] G. Holker and M. V. dos Santos, "Toward an estimation of distribution algorithm for the evolution of artificial neural networks," in *Proceedings of the Third C* Conference on Computer Science and Software Engineering*. ACM, 2010, pp. 17–22.

[9] N. Hansen and A. Ostermeier, "Completely derandomized self-adaptation in evolution strategies," *Evolutionary computation*, vol. 9, no. 2, pp. 159–195, 2001.

[10] C. Igel, "Neuroevolution for reinforcement learning using evolution strategies," in *The 2003 Congress on Evolutionary Computation, 2003. CEC'03.*, vol. 4. IEEE, 2003, pp. 2588–2595.

[11] R. Wang, J. Clune, and K. O. Stanley, "Vine: an open source interactive data visualization tool for neuroevolution," in *Proceedings of the Genetic and Evolutionary Computation Conference Companion*. ACM, 2018, pp. 1562–1564.

[12] A. Liaw, M. Wiener *et al.*, "Classification and regression by randomforest," *R news*, vol. 2, no. 3, pp. 18–22, 2002.

[13] S. Mussmann and P. Liang, "Generalized binary search for split-neighborly problems," *arXiv preprint arXiv:1802.09751*, 2018.

[14] Y.-C. Chang, "N-dimension golden section search: Its variants and limitations," in *2009 2nd International Conference on Biomedical Engineering and Informatics*. IEEE, 2009, pp. 1–6.

[15] J. A. Koupaei, S. M. M. Hosseini, and F. M. Ghaini, "A new optimization algorithm based on chaotic maps and golden section search method," *Engineering Applications of Artificial Intelligence*, vol. 50, pp. 201–214, 2016.

[16] J. Guillot, D. Restrepo-Leal, C. Robles-Algarín, and I. Oliveros, "Search for global maxima in multimodal functions by applying numerical optimization algorithms: a comparison between golden section and simulated annealing," *Computation*, vol. 7, no. 3, p. 43, 2019.

[17] S. Henikoff and J. G. Henikoff, "Position-based sequence weights," *Journal of molecular biology*, vol. 243, no. 4, pp. 574–578, 1994.

[18] J. Kiefer, "Sequential minimax search for a maximum," *Proceedings of the American mathematical society*, vol. 4, no. 3, pp. 502–506, 1953.

[19] R. W. Emerson, "Causation and pearson's correlation coefficient," *Journal of visual impairment & blindness*, vol. 109, no. 3, pp. 242–244, 2015.

[20] A. Saxena, M. Prasad, A. Gupta, N. Bharill, O. P. Patel, A. Tiwari, M. J. Er, W. Ding, and C.-T. Lin, "A review of clustering techniques and developments," *Neurocomputing*, vol. 267, pp. 664–681, 2017.

[21] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," *arXiv preprint arXiv:1606.01540*, 2016.

[22] J. Reisinger, K. O. Stanley, and R. Miikkulainen, "Evolving reusable neural modules," in *Genetic and Evolutionary Computation Conference*. Springer, 2004, pp. 69–81.

[23] T. Watts, B. Xue, and M. Zhang, "Blocky net: A new neuroevolution method," in *2019 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2019, pp. 586–593.

[24] J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.

[25] R. Southwell, J. Huang, and C. Cannings, "Complex networks from simple rewrite systems," *arXiv preprint arXiv:1205.0596*, 2012.

[26] E. Tanyildizi, "A novel optimization method for solving constrained and unconstrained problems: modified golden sine algorithm," *Turkish Journal of Electrical Engineering & Computer Sciences*, vol. 26, no. 6, pp. 3287–3304, 2018.

[27] A. Aurasopon and W. Khamsen, "An improved local search involving bee colony optimization using lambda iteration combined with a golden section search method to solve an economic dispatch problem," *Przeglad Elektrotechniczny*, 2019.

[28] S. Das, S. Maity, B.-Y. Qu, and P. N. Suganthan, "Real-parameter evolutionary multimodal optimizationa survey of the state-of-the-art," *Swarm and Evolutionary Computation*, vol. 1, no. 2, pp. 71–88, 2011.

[29] X. Wang, Y. Wang, H. Wu, L. Gao, L. Luo, P. Li, and X. Shi, "Fibonacci multi-modal optimization algorithm in noisy environment," *Applied Soft Computing*, p. 105874, 2019.

[30] J. E. Auerbach and J. C. Bongard, "Evolving complete robots with cppn-neat: the utility of recurrent connections," in *Proceedings of the 13th annual conference on Genetic and evolutionary computation*. ACM, 2011, pp. 1475–1482.

[31] J. Huizinga, J.-B. Mouret, and J. Clune, "Does aligning phenotypic and genotypic modularity improve the evolution of neural networks?" in *Proceedings of the Genetic and Evolutionary Computation Conference 2016*. ACM, 2016, pp. 125–132.

[32] K. O. Stanley, "Compositional pattern producing networks: A novel abstraction of development," *Genetic programming and evolvable machines*, vol. 8, no. 2, pp. 131–162, 2007.

[33] F. Gruau, "Genetic synthesis of boolean neural networks with a cell rewriting developmental process," in *[Proceedings] COGANN-92: International Workshop on Combinations of Genetic Algorithms and Neural Networks*. IEEE, 1992, pp. 55–74.

[34] S. Luke and L. Spector, "Evolving graphs and networks with edge encoding: Preliminary report," in *Late breaking papers at the genetic programming 1996 conference*. Citeseer, 1996, pp. 117–124.

[35] N. R. Brisaboa, S. Ladra, and G. Navarro, "k 2-trees for compact web graph representation," in *International Symposium on String Processing and Information Retrieval*. Springer, 2009, pp. 18–30.

[36] Y. Jin and B. Sendhoff, "Reducing fitness evaluations using clustering techniques and neural network ensembles," in *Genetic and Evolutionary Computation Conference*. Springer, 2004, pp. 688–699.

[37] V. Maniezzo, "Genetic evolution of the topology and weight distribution of neural networks," *IEEE Transactions on neural networks*, vol. 5, no. 1, pp. 39–53, 1994.

[38] H. H. Hoos and T. Stützle, *Stochastic local search: Foundations and applications*. Elsevier, 2004.

[39] H. Anton and C. Rorres, *Elementary Linear Algebra, Binder Ready Version: Applications Version*. John Wiley & Sons, 2013.

[40] O. Bachem, M. Lucic, S. H. Hassani, and A. Krause, "Approximate k-means++ in sublinear time," in *Thirtieth AAAI Conference on Artificial Intelligence*, 2016.

[41] D. Yan, L. Huang, and M. I. Jordan, "Fast approximate spectral clustering," in *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*. ACM, 2009, pp. 907–916.

[42] T. Zhang, R. Ramakrishnan, and M. Livny, "Birch: an efficient data clustering method for very large databases," in *ACM Sigmod Record*, vol. 25, no. 2. ACM, 1996, pp. 103–114.

[43] C.-H. H. Yang, J. Qi, P.-Y. Chen, Y. Ouyang, I. Hung, T. Danny, C.-H. Lee, and X. Ma, "Enhanced adversarial strategically-timed attacks against deep reinforcement learning," *arXiv preprint arXiv:2002.09027*, 2020.